Лабораторна робота №1. Моделювання предметної області та створення схеми бази даних

Мета та очікувані результати

- Мета: пройти для власної предметної області весь шлях від опису бізнесу до робочої схеми в PostgreSQL: бізнес-правила, ER-модель, ключі, нормалізація до третьої нормальної форми, реляційна схема і міграція, яка створює таблиці з ключами й обмеженнями цілісності і вміє їх прибрати. Схема цієї роботи - основа всіх наступних робіт курсу.
- Після виконання роботи студент може: сформулювати бізнес-правила предметної області і для кожного вказати, чи перевіряє його схема бази і яким обмеженням; побудувати ER-модель у Mermaid із кардинальністю та обов'язковістю участі для кожного зв'язку; обґрунтувати вибір первинного ключа і реакції на видалення; показати три аномалії на «поганій» таблиці і привести її до 3NF; перетворити ER-модель на реляційну схему з англійськими назвами таблиць і стовпців за п'ятьма правилами переходу; запустити PostgreSQL у контейнері за файлом опису середовища і підключитися до нього pgAdmin; написати міграцію - файли up і down - і показати, що СУБД відхиляє дані, які порушують обмеження; відтворити схему в порожній базі лише з файлів репозиторію і прибрати її зворотною міграцією.
- Інструменти: Docker із файлом опису середовища і pgAdmin - робоче середовище PostgreSQL; Mermaid - діаграми; Git - здача.
- Що свідомо не входить у роботу: наповнення таблиць даними, вибірка, транзакції й одночасний доступ - робота №2; індекси - робота №3.

Словник термінів роботи

Нових термінів робота не вводить: усі терміни введено на лекціях 1-5. Приклади взято з прокату велосипедів; у звіті відповідні приклади наводяться для власної предметної області.

- **Предметна область** - обмежена частина реального світу, про яку система зберігає дані: її об'єкти, події та правила. Приклад: область, обрана на першому тижні й описана в README репозиторію, - та сама для всіх п'яти робіт. Повторення з лекції 1.
- **Наскрізний проєкт** - формат практикуму: усі п'ять лабораторних робіт виконуються на одній предметній області, і кожна наступна робота розширює результат попередньої. Приклад: таблиці, створені в цій роботі, у роботі №2 наповнюються даними. Повторення з лекції 1.
- **Правило** (бізнес-правило) - твердження вигляду «якщо умова - то дія або висновок», яке визначає, як організація діє в певній ситуації. Приклад: «якщо за пунктом закріплено хоча б один велосипед, пункт не можна вилучити з обліку». Повторення з лекції 1.
- **Репозиторій** (Git) - каталог проєкту під контролем версій, у якому зберігається історія всіх змін; у курсі - єдиний спосіб здачі робіт. Приклад: коміт із файлами міграції показує, коли і

в якому вигляді схему створено вперше. Повторення з лекції 1.

- **Система керування базами даних** (СУБД) - програмний комплекс, який зберігає дані, надає доступ до них, перевіряє правила, обслуговує одночасну роботу багатьох користувачів і відновлює дані після збою. Приклад: PostgreSQL 18, запущений у контейнері за файлом опису цієї роботи. Повторення з лекції 2.
- **Архітектура клієнт-сервер** - схема роботи, у якій СУБД (сервер) обслуговує запити багатьох програм-клієнтів; клієнт не має прямого доступу до файлів даних. Приклад: pgAdmin на комп'ютері студента - клієнт, PostgreSQL у контейнері - сервер. Повторення з лекції 2.
- **Образ** (Docker image) - готовий незмінний шаблон програми разом з її оточенням; з нього створюються контейнери. Приклад: `postgres:18.6` - образ, зазначений у файлі опису середовища. Повторення з лекції 2.
- **Контейнер** - запущений екземпляр образу: ізольований процес зі власною файловою системою. Приклад: після команди `docker compose up -d` команда `docker compose ps` показує запущений контейнер PostgreSQL. Повторення з лекції 2.
- **Том** (volume) - окреме сховище, підключене до контейнера, у якому дані переживають перезапуск і видалення контейнера. Приклад: після `docker compose down` таблиці бази лишаються в томі, а `docker compose down -v` видаляє том разом з усіма даними. Повторення з лекції 2.
- **Відображення порту** - правило «звертання до порту на моєму комп'ютері передається в контейнер». Приклад: рядок `"5432:5432"` у файлі опису; якщо порт 5432 на комп'ютері вже зайнятий, ліве число замінюють на 5433 і вказують цей порт у pgAdmin. Повторення з лекції 2.
- **Файл опису середовища** (docker compose) - текстовий файл, у якому перелічено, які контейнери запустити, з якими образами, томами й портами; запускається однією командою. Приклад: `compose.yaml` у корені репозиторію студента. Повторення з лекції 2.
- **Схема даних** - опис структури бази: які є таблиці, які в них стовпці, якого типу і які правила діють. Приклад: результат скрипта `check-schema.sql` - перелік таблиць, стовпців і обмежень схеми. Повторення з лекції 2.
- **Метадані** (системний каталог) - дані про дані: службові таблиці самої СУБД, у яких зберігається схема. Приклад: скрипт `check-schema.sql` читає не рядки таблиць студента, а системний каталог. Повторення з лекції 2.
- **Таблиця** (відношення) - множина рядків однакової структури, що зберігає дані про однотипні об'єкти. Приклад: таблиця `tariffs`. Повторення з лекції 2.
- **Стовпець** (поле, атрибут) - одна характеристика, що зберігається для всіх рядків таблиці. Приклад: `price` у таблиці `tariffs`. Повторення з лекції 2.
- **Тип даних** - обмеження на множину допустимих значень стовпця. У ER-моделі - загальні типи (текст, число, дата, дата_час, логічне), у реляційній схемі й міграції - типи PostgreSQL (`text`, `integer`, `numeric(p, s)`, `date`, `timestamp`, `boolean`). Приклад: дата придбання велосипеда - `purchase_date date`. Повторення з лекцій 2 і 5.
- **Обмеження** (constraint) - правило, записане в схемі й перевірене самою СУБД. Приклад: оцінку стану 7 СУБД не збереже, бо в схемі записано умову «від 1 до 5». Повторення з лекції 2.

- **Концептуальна модель** - опис предметної області в поняттях самої області, без огляду на конкретну СУБД. Приклад: ER-діаграма роботи з українськими назвами і загальними типами, яку можна обговорити із замовником без знання SQL. Повторення з лекції 3.
- **Логічна модель** - той самий опис, перекладений у структури обраної моделі даних (для реляційної - таблиці, стовпці, ключі), але без деталей фізичного зберігання. Приклад: перелік таблиць після нормалізації з первинними й зовнішніми ключами. Повторення з лекції 3.
- **Фізична модель** - опис того, як дані реально зберігаються в конкретній СУБД: типи, індекси, розміщення. Приклад: реляційна схема з англійськими назвами і типами PostgreSQL та файл міграції, що її створює. Повторення з лекцій 3 і 5.
- **Модель «сутність-зв'язок»** (ER-модель, Entity-Relationship) - спосіб зображення предметної області у вигляді сутностей, їхніх атрибутів і зв'язків між ними; EER додає узагальнення. Приклад узагальнення: «клієнт» із підтипами «приватна особа» і «організація», у кожного з яких є власні атрибути. Повторення з лекції 3.
- **Нотація діаграми** - спосіб позначення на схемі. У курсі - Crow's foot («воронячі лапки»). Приклад: у Mermaid запис `||--o{` читається «рівно один - нуль або багато». Повторення з лекції 3.
- **Сутність** - тип об'єкта або події, про який зберігаються однотипні дані. Приклад: «Тариф». Повторення з лекції 3.
- **Атрибут** - окрема характеристика сутності. Приклад: у сутності «Тариф» - назва і ціна. Повторення з лекції 3.
- **Домен атрибута** - множина допустимих значень: тип плюс додаткові обмеження. Приклад: ціна тарифу - число з двома знаками після коми, більше нуля. Повторення з лекції 3.
- **Складений атрибут** і **багатозначний атрибут** - атрибут, що розкладається на частини, і атрибут, що має кілька значень одночасно; багатозначний атрибут у реляційній моделі перетворюється на окрему таблицю. Приклад: кілька адрес електронної пошти клієнта. Повторення з лекції 3.
- **Ідентифікатор сутності** - атрибут або набір атрибутів, що однозначно вирізняє екземпляр серед інших. Приклад: код тарифу. Повторення з лекції 3.
- **Зв'язок** - логічне відношення між сутностями, яке відображає факт предметної області. Приклад: «тариф застосовується до видачі». Повторення з лекції 3.
- **Кардинальність зв'язку** - скільки екземплярів однієї сутності може відповідати одному екземпляру іншої: 1:1, 1:N, M:N. Приклад: один тариф - багато видач (1:N). Повторення з лекції 3.
- **Обов'язковість участі** - чи може екземпляр існувати без зв'язку. Приклад: пункт може існувати без жодного велосипеда, а велосипед без пункту - ні. Повторення з лекції 3.
- **Асоціативна сутність** (сутність-зв'язок) - сутність, у яку перетворюють зв'язок «багато до багатьох», особливо коли він має власні атрибути. Приклад: зв'язок «велосипед - механік» із датою і вартістю робіт стає сутністю «Ремонт». Повторення з лекції 3.
- **Слабка сутність** - сутність, яка не має самостійного ідентифікатора й існує лише в межах іншої. Приклад: «етап ремонту» існує тільки в межах конкретного ремонту. Повторення з лекції 3.
- **Потенційний ключ** (candidate key) - набір атрибутів, що однозначно ідентифікує рядок і не містить надлишкових атрибутів. Приклад: у велосипеда два потенційні ключі - інвентарний

номер і номер рами. Повторення з лекції 3.

- **Первинний ключ** - той потенційний ключ, який обрано основним засобом ідентифікації; він обов'язковий і унікальний. Приклад: інвентарний номер велосипеда. Повторення з лекції 3.
- **Природний ключ** і **сурогатний (штучний) ключ** - ключ із реальних даних предметної області і службовий ідентифікатор без змісту, створений системою. Правило курсу: сурогатний ключ обирають тоді, коли природний може змінитися або не гарантує унікальності. Приклад: службовий номер пункту замість назви, яку можуть змінити. Повторення з лекції 3.
- **Зовнішній ключ** - атрибут, значення якого має відповідати первинному ключу рядка в іншій таблиці; саме він реалізує зв'язок. Приклад: `tariff_code` у таблиці `rentals`. Повторення з лекції 3.
- **Цілісність сутностей** - вимога, щоб первинний ключ був завжди заповнений і унікальний. Приклад: два тарифи з кодом Г1 в одній таблиці неможливі. Повторення з лекції 3.
- **Посилальна цілісність** - вимога, щоб зовнішній ключ указував на рядок, який справді існує. Приклад: велосипед не можна закріпити за пунктом, якого немає в таблиці `stations`. Повторення з лекції 3.
- **Реакція на видалення** - правило, що застосовується, коли видаляють рядок, на який є посилання: заборонити, видалити разом (каскадно) або обнулити посилання. Вибір реакції - рішення моделювальника, а не технічна деталь. Приклад: пункт, за яким закріплено велосипеди, видаляти заборонено. Повторення з лекції 3.
- **Значення NULL** - позначка «значення відсутнє або невідоме», а не нуль і не порожній рядок; для первинного ключа неприпустиме. Приклад: телефон клієнта, який його не повідомив. Повторення з лекції 3.
- **Надмірність даних** - повторення тих самих відомостей у багатьох рядках; сама по собі не помилка, але джерело аномалій. Приклад: назва тарифу, записана в кожному рядку видачі. Повторення з лекції 4.
- **Аномалія оновлення** - ситуація, коли одну й ту саму зміну доводиться вносити в багато рядків, і частина з них може залишитися застарілою. Приклад: тариф перейменували, а назву виправили лише в частині видач. Повторення з лекції 4.
- **Аномалія вставки** - неможливість зберегти факт через відсутність іншого, не пов'язаного з ним. Приклад: новий тариф неможливо внести, поки за ним не видали жодного велосипеда. Повторення з лекції 4.
- **Аномалія видалення** - втрата потрібних відомостей як побічний наслідок видалення інших. Приклад: видалили єдину видачу за тарифом - зникли відомості про сам тариф. Повторення з лекції 4.
- **Функціональна залежність** (X → Y) - ситуація, коли значення набору атрибутів X однозначно визначає значення Y: «знаючи X, я завжди знаю Y». Приклад: номер рами → модель велосипеда. Повторення з лекції 4.
- **Повна функціональна залежність** - залежність від усього складеного ключа, а не від його частини. **Часткова залежність** - залежність лише від частини складеного ключа. Приклад часткової: у таблиці видач із ключем (інвентарний номер, дата й час видачі) номер рами залежить лише від інвентарного номера. Повторення з лекції 4.
- **Транзитивна залежність** - залежність через посередника: ключ визначає атрибут А, а А визначає Б. Приклад: ключ видачі визначає клієнта, клієнт визначає телефон. Повторення з

лекції 4.

- **Нормалізація** - послідовне розкладання таблиць на менші так, щоб усунути аномалії, зберігши всі дані та зв'язки між ними. Приклад: винесення тарифу з «поганої» таблиці видач в окрему таблицю. Повторення з лекції 4.
- **Перша нормальна форма** (1NF) - у кожній комірці одне неподільне значення, немає повторюваних груп стовпців. Приклад порушення: стовпці «email1», «email2». Повторення з лекції 4.
- **Друга нормальна форма** (2NF) - таблиця в 1NF, і кожен неключовий атрибут залежить від усього первинного ключа; має сенс лише для складених ключів. Приклад: номер рами виносять із таблиці видач із ключем (інвентарний номер, дата й час видачі) у таблицю велосипедів. Повторення з лекції 4.
- **Третя нормальна форма** (3NF) - таблиця в 2NF без транзитивних залежностей. Приклад: телефон клієнта виносять із таблиці видач у таблицю клієнтів. Повторення з лекції 4.
- **Денормалізація** - свідоме повернення надмірності заради швидкості читання з контролем ризику неузгодженості. Приклад: збережена кількість велосипедів на пункті поряд із таблицею велосипедів. Повторення з лекції 4.
- **Складений ключ** - первинний ключ із двох і більше атрибутів. Приклад: `PRIMARY KEY (inventory_number, rented_at)`. Повторення з лекції 4.
- **Таблиця зв'язку** - таблиця, що реалізує зв'язок «багато до багатьох» і містить зовнішні ключі на обидві сторони. Приклад: таблиця `repairs` із ключами велосипеда і механіка. Повторення з лекції 4.
- **Правила переходу від ER-моделі до реляційної схеми** - сутність стає таблицею; зв'язок 1:N - зовнішнім ключем на боці «багато»; M:N - окремою таблицею; багатозначний атрибут - окремою таблицею; слабка сутність - складеним ключем із ключем власника. Приклад: зв'язок «пункт - велосипед» дає стовпець `station_name` у таблиці `bikes`. Повторення з лекції 4.
- **SQL** (мова структурованих запитів) - мова, якою описують структуру реляційної бази, змінюють дані та отримують їх. Приклад: файли міграції цієї роботи містять команди SQL. Повторення з лекції 5.
- **Команди визначення даних** (DDL) - частина SQL, яка створює, змінює і видаляє об'єкти схеми: CREATE, ALTER, DROP. Приклад: усі команди файлів `001_create_schema.up.sql` і `001_create_schema.down.sql`. Повторення з лекції 5.
- **CREATE TABLE** (створити таблицю) - команда, яка додає до схеми нову таблицю зі стовпцями, типами й обмеженнями. Приклад: `CREATE TABLE tariffs (tariff_code text PRIMARY KEY, tariff_name text NOT NULL, price numeric(8, 2) NOT NULL CHECK (price > 0));`. Повторення з лекції 5.
- **DROP TABLE** (видалити таблицю) - команда, яка видаляє таблицю разом з усіма її даними. Приклад: `DROP TABLE bikes;` у файлі down виконується раніше за `DROP TABLE stations;`, бо велосипеди посилаються на пункти. Повторення з лекції 5.
- **PRIMARY KEY**, **NOT NULL**, **UNIQUE**, **CHECK** - обмеження в команді: первинний ключ; обов'язкове значення; значення не повторюється; умова, яку задовольняє кожен рядок. Приклад: `frame_number text NOT NULL UNIQUE`. Повторення з лекції 5.
- **REFERENCES** і **ON DELETE** - обмеження, яке реалізує зовнішній ключ, і його частина, що задає реакцію на видалення: RESTRICT - заборонити, CASCADE - видалити разом, SET

NULL - обнулити посилання. Приклад: `tariff_code text NOT NULL REFERENCES tariffs (tariff_code) ON DELETE RESTRICT`. Повторення з лекції 5.

- **Стовпець ідентичності** (identity column) - стовпець, значення якого СУБД видає сама за лічильником; так реалізують сурогатний ключ. Приклад: номер в електронній черзі видає термінал, а не відвідувач; так само `station_id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY`. Повторення з лекції 5.
- **ALTER TABLE** (змінити таблицю) - команда, яка змінює наявну таблицю, не втрачаючи її даних. Приклад: `ALTER TABLE customers ADD COLUMN email text;`. Повторення з лекції 5.
- **Файл міграції** (migration) - файл із командами однієї зміни схеми; файли виконуються по черзі за номером, а вже виконаний файл не редагують. Приклад: помилку в закоміченій міграції 001 виправляють міграцією 002. Повторення з лекції 5.
- **Зворотна міграція** (down migration) - файл, який точно скасовує зміну схеми, виконану парним файлом up, і повертає схему до стану перед ним; команди в ньому йдуть у зворотному порядку. Приклад: складені в коробку речі дістають у зворотному порядку; так само `001_create_schema.down.sql` видаляє таблиці від `rentals` до `stations`. Повторення з лекції 5.
- **INSERT** (додати) - команда, яка додає рядки до таблиці. У цій роботі - лише для перевірки обмежень: кожна перевірочна команда має бути відхилена СУБД. Приклад: `INSERT INTO tariffs (tariff_code, tariff_name, price) VALUES ('Г1', 'Погодинний', -60);`. Повторення з лекції 5.

Зв'язок з лекціями

Роботу виконують після лекції 5. Завдання для самостійної роботи лекцій 1-5 - чернетки кроків цієї роботи: їх доопрацьовують за вимогами нижче, а не створюють заново.

Лекція	Що потрібно для роботи	Чернетка із завдання для самостійної роботи
1	предметна область, бізнес-правило	README репозиторію з описом області й переліком сутностей
2	СУБД, реляційна модель, контейнер, файл опису середовища	запуск PostgreSQL і протокол запуску; три правила «схема чи програма»
3	ER-модель, ключі, цілісність, реакція на видалення	ER-модель у Mermaid; таблиця ключів; реакції на видалення; перевірка за чотирма типовими помилками
4	функціональні залежності, нормалізація, правила переходу	залежності X → Y; «погана» таблиця і кроки нормалізації; перелік таблиць
5	CREATE TABLE, обмеження, міграції up і down, англійські назви структури	міграція 001 (up і down) і цикл up - down - up

Підготовка середовища

Потрібні три програми: Docker із Docker Compose, pgAdmin 4 версії 9.17 як застосунок для комп'ютера і Git. Встановлення виконується один раз і не входить у час заняття.

Windows

1. Перевірити, що апаратна віртуалізація увімкнена: Диспетчер завдань - Продуктивність - ЦП, рядок «Віртуалізація: Увімкнено» (в англійському інтерфейсі - Task Manager - Performance - CPU - Virtualization: Enabled). Якщо вимкнено, її вмикають у налаштуваннях BIOS/UEFI комп'ютера (пункт Intel VT-x або AMD-V / SVM).

2. Встановити WSL 2 - підсистему Linux, на якій працює Docker Desktop. Відкрити PowerShell від імені адміністратора і виконати:

   ```
   wsl --install --no-distribution
   ```

   Після завершення перезавантажити комп'ютер. Якщо WSL уже встановлено, достатньо оновити його командою `wsl --update`.

3. Завантажити Docker Desktop for Windows з офіційного сайту Docker і встановити, залишивши позначку використання WSL 2 замість Hyper-V (Use WSL 2 instead of Hyper-V). Вимоги до версії Windows і обсягу пам'яті - у документації Docker, розділ Get started, сторінка встановлення Docker Desktop on Windows.

4. Запустити Docker Desktop і дочекатися, поки внизу вікна з'явиться стан «Engine running». Docker Desktop має бути запущений щоразу, коли потрібна база: без нього команди `docker` не працюють.

5. У звичайному (не адміністраторському) вікні PowerShell або Windows Terminal перевірити встановлення:

   ```
   docker version
   docker compose version
   ```

   Команда `docker version` має вивести два розділи - Client і Server. Якщо розділу Server немає, Docker Desktop не запущений.

6. Встановити pgAdmin 4 версії 9.17 окремим інсталятором pgAdmin для Windows. Не встановлювати PostgreSQL з інсталятора для Windows заради pgAdmin: такий інсталятор додає ще й локальний сервер PostgreSQL, який займає порт 5432 і заважає контейнеру.

7. Встановити Git for Windows.

8. Усі файли роботи (SQL-скрипти, звіт) зберігати в кодуванні UTF-8; редактор VS Code робить це за замовчуванням. В іншому кодуванні українські значення в рядках даних і коментарях відображаються пошкодженими.

macOS і Linux

- macOS: Docker Desktop for Mac (збірка для процесора Apple або Intel відповідно до комп'ютера) і pgAdmin 4 версії 9.17 для macOS. Після встановлення запустити Docker Desktop і перевірити командами `docker version` і `docker compose version` у Terminal.
- Linux: Docker Engine із плагіном Docker Compose за документацією Docker для свого дистрибутива і pgAdmin 4 версії 9.17 у варіанті для комп'ютера (desktop). Користувача слід додати до групи `docker`, щоб команди виконувалися без `sudo`.

Запуск і підключення

1. Скопіювати файл опису середовища (`lab-01/compose.yaml`) в корінь свого репозиторію під назвою `compose.yaml`. Файл спільний для робіт курсу, тому лежить поза каталогом роботи.
2. У терміналі (у Windows - PowerShell або Windows Terminal) перейти в корінь репозиторію командою `cd` і виконати:

   ```
   docker compose up -d
   docker compose ps
   ```

   Перший запуск завантажує образ `postgres:18.6`, тому потребує мережі. Контейнер має бути в стані `running`.
3. У pgAdmin зареєструвати сервер (Register - Server): на вкладці General - довільна назва, на вкладці Connection - сервер `localhost`, порт `5432`, база `dbsa`, користувач `student`, пароль `student`.
4. Відкрити Query Tool для бази `dbsa` і переконатися, що підключення працює: у дереві об'єктів розкривається база `dbsa` зі схемою `public`, у якій ще немає таблиць.
5. Команди запуску і вивід `docker compose ps` записати в протокол запуску (розділ 7 звіту).

Після перезавантаження комп'ютера контейнер сам не запускається: спершу запускають Docker Desktop, потім у корені репозиторію знову виконують `docker compose up -d`. Дані при цьому зберігаються в томі.

Якщо Docker не запускається на власному комп'ютері, до викладача звертаються до початку роботи, а не в день здачі: як працювати в такому разі, оголошує викладач.

Порядок виконання

Крок 1. Предметна область і бізнес-правила

- Уточнити опис предметної області з README репозиторію: 6-8 сутностей, не більше 10; схема має вміщуватися на один аркуш.
- Сформулювати не менше восьми бізнес-правил у формі «якщо - то».
- Для кожного правила вказати, де його перевіряють: у схемі - яким обмеженням (первинний ключ, NOT NULL, UNIQUE, CHECK, REFERENCES з ON DELETE) або в програмі - з

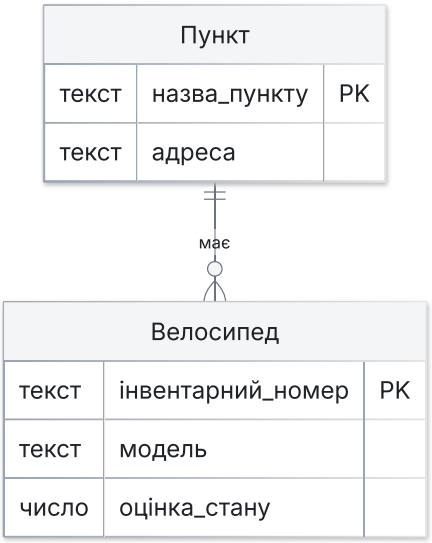
коротким обґрунтуванням. Схемою перевіряється щонайменше п’ять правил.

Результат: розділ 2 звіту - таблиця «правило - де перевіряється - обмеження або обґрунтування».

Крок 2. ER-модель

- Виділити сутності, атрибути із загальними типами (текст, число, дата, дата_час, логічне) та ідентифікатори. ER-модель - концептуальна, тому назви в ній українською, мовою предметної області.
- Для кожного зв’язку визначити кардинальність і обов’язковість участі та сформулювати його реченням в обидва боки: «один пункт має багато велосипедів - кожен велосипед закріплений за одним пунктом».
- Розкласти кожен зв’язок «багато до багатьох» в асоціативну сутність. Якщо у зв’язку є власний атрибут (дата, кількість, ціна), це вже сутність.
- Намалювати діаграму `erDiagram` у Mermaid. Назви сутностей - кирилицею; атрибут записується як `тип назва_атрибута`, слова назви з’єднуються підкресленням; ідентифікатор позначається `PK`; зв’язок підписується дієсловом. Позначення кардинальності з лівого боку зв’язку: `||` - рівно один, `|o` - нуль або один, `}|` - один або багато, `}o` - нуль або багато; з правого боку ті самі позначення дзеркальні: `||`, `o|`, `|{`, `o{`.

```
erDiagram
    Пункт ||--o{ Велосипед : "має"
    Пункт {
        текст назва_пункту PK
        текст адреса
    }
    Велосипед {
        текст інвентарний_номер PK
        текст модель
        число оцінка_стану
    }
```



- Узагальнення (EER) використовується лише за потреби. Mermaid не має для нього окремого позначення, тому узагальнення описують текстом під діаграмою: загальна сутність, підтипи і їхні власні атрибути.
- Перевірити модель за чотирма типовими помилками лекції 3: атрибут, який насправді є сутністю; зв’язок без кардинальності; одна таблиця на багато стовпців замість кількох сутностей; телефон або електронна пошта як первинний ключ.

Результат: розділ 3 звіту - діаграма, перелік сутностей (одне речення на сутність), зв’язки реченнями в обидва боки, відмітка про перевірку за типовими помилками.

Крок 3. Ключі та реакції на видалення

- Для кожної сутності виписати потенційні ключі, обрати первинний і одним реченням обґрунтувати вибір природного або сурогатного ключа за правилом лекції 3.
- Для кожного зв’язку «один до багатьох» обрати реакцію на видалення (заборонити, видалити разом, обнулити посилання) і обґрунтувати її наслідком для бізнесу, а не технікою. Реакція «обнулити посилання» можлива лише тоді, коли зовнішній ключ допускає NULL.

Результат: розділ 4 звіту - таблиця ключів і таблиця реакцій на видалення.

Крок 4. Нормалізація

- Виписати не менше восьми функціональних залежностей у вигляді X → Y.
- Побудувати «погану» широку таблицю власної області (одна таблиця, щонайменше десять стовпців, 3-5 рядків прикладу) і показати на ній аномалії оновлення, вставки й видалення конкретними діями.
- Привести таблицю до 1NF, 2NF і 3NF. Після кожного кроку записати отримані таблиці з ключами і проблему, яку знято.
- Звірити таблиці після 3NF з ER-моделлю кроку 2 і пояснити розбіжності, якщо вони є. Розбіжність означає помилку в одному з двох результатів.
- Свідому денормалізацію, якщо вона є, описати окремим абзацом: що дублюється, навіщо і хто підтримує узгодженість. Без такого обґрунтування таблиця має бути в 3NF.

Результат: розділ 5 звіту.

Крок 5. Реляційна схема

- За п'ятьма правилами переходу перетворити ER-модель на перелік таблиць зі стовпцями, типами PostgreSQL, первинними й зовнішніми ключами.
- Назвати таблиці, стовпці й обмеження так, як їх буде створено в базі. Усе, що належить до структури бази, - англійською: малими латинськими літерами, слова через підкреслення, таблиці в множині (`stations`, `bikes`, `rentals`), стовпці - в одиничному числі (`station_name`, `purchase_date`). Назви - англійські слова, а не транслітерація (`customers`, а не `klienty`), без лапок і великих літер. Вміст рядків даних може бути українською.
- Намалювати реляційну схему діаграмою `erDiagram`: назви таблиць і стовпців - точно як у SQL; типи PostgreSQL без параметрів (`numeric`, а не `numeric(8, 2)`, бо Mermaid не допускає коми в типі); ключі - `PK`, `FK`, для ключа, що є і первинним, і зовнішнім, - `PK, FK`; кожен атрибут складеного ключа позначається `PK`. Зразок - слайд «Схема прокату в PostgreSQL» лекції 5.

Результат: розділ 6 звіту.

Крок 6. Міграція 001: файли up і down

- Створити в каталозі `lab-01/migrations` два файли однієї міграції:
  - `001_create_schema.up.sql` - команди CREATE TABLE для всіх таблиць схеми;
  - `001_create_schema.down.sql` - команди DROP TABLE, які видаляють ці таблиці і повертають базу до стану перед up.
- Кожен файл починається коментарем, що пояснює, яку зміну він виконує або скасовує.
- У файлі up спершу створюються таблиці, на які посилаються, потім ті, що посилаються. У файлі down порядок зворотний: спершу видаляються таблиці, що посилаються, останніми - ті, на які посилаються.
- Кожна таблиця має первинний ключ. Кожен зовнішній ключ записується через REFERENCES із переліком стовпців і явним ON DELETE, обраним на кроці 3. Обмеження NOT NULL, UNIQUE і CHECK відповідають таблиці бізнес-правил кроку 1.

```sql
-- 001_create_schema.up.sql
-- Створення схеми прокату: пункти і велосипеди (фрагмент для зразка оформлення).

CREATE TABLE stations (
    station_name text PRIMARY KEY,
    address      text NOT NULL,
    city         text NOT NULL
);

CREATE TABLE bikes (
    inventory_number text PRIMARY KEY,
    frame_number     text NOT NULL UNIQUE,
    model            text NOT NULL,
    purchase_date    date NOT NULL,
    condition_rating integer NOT NULL CHECK (condition_rating >= 1 AND condition_rating <=
5),
    station_name     text NOT NULL REFERENCES stations (station_name) ON DELETE RESTRICT
);
```

```sql
-- 001_create_schema.down.sql
-- Скасування 001_create_schema.up.sql: таблиці видаляються у зворотному порядку.

DROP TABLE bikes;
DROP TABLE stations;
```

- Виконати файл up у pgAdmin: Query Tool для бази `dbsa`, відкрити файл, виконати. Перевірити назву бази в заголовку Query Tool: таблиці мають з'явитися в базі `dbsa`, а не в службовій базі `postgres`.
- Поки файли не потрапили в коміт, їх можна виправляти; перевіряти виправлені файли слід на порожній базі (крок 9). Після коміту файли міграції не редагують: будь-яку зміну схеми, у тому числі виправлення за зауваженнями викладача, оформлюють наступною міграцією - парою файлів `002_опис.up.sql` і `002_опис.down.sql` з командами ALTER TABLE.

Результат: два файли міграції і розділ 8 звіту - рішення, яких не видно з діаграми (умови CHECK, реакції на видалення, стовпці ідентичності).

Крок 7. Перевірка обмежень

- Створити файл `lab-01/constraint-checks.sql` щонайменше з п'ятьма командами, кожна з яких порушує одне обмеження схеми: NOT NULL, UNIQUE, CHECK, REFERENCES і первинний ключ. Перед кожною командою - коментар із правилом, яке перевіряється, і очікуваним повідомленням СУБД.
- Кожна перевірка - одна команда, яку СУБД відхиляє повністю, тож база лишається порожньою. Порушення первинного ключа чи UNIQUE показують однією командою INSERT

із двома рядками з однаковим значенням; порушення REFERENCES - рядком, що посилається на неіснуючий рядок, коли решта значень коректні.

```
-- Правило 3: оцінка стану велосипеда - від 1 до 5 (CHECK).
-- Очікувано: violates check constraint "bikes_condition_rating_check"
INSERT INTO bikes (inventory_number, frame_number, model, purchase_date, condition_rating,
station_name)
VALUES ('B-27', 'FR-1027', 'City', '2023-04-12', 7, 'Пункт №4');

-- Правило 5: номер рами не повторюється (UNIQUE).
-- Очікувано: duplicate key value violates unique constraint "bikes_frame_number_key"
INSERT INTO bikes (inventory_number, frame_number, model, purchase_date, condition_rating,
station_name)
VALUES ('B-27', 'FR-1027', 'City', '2023-04-12', 4, 'Пункт №4'),
       ('B-31', 'FR-1027', 'Trek', '2023-05-03', 5, 'Пункт №4');
```

- У pgAdmin кожну команду виконують окремо: виділити команду разом із коментарем і виконати. Під час виконання всього файлу поспіль виконання зупиняється на першій помилці.
- Для кожної перевірки записати в звіт правило, вид обмеження і перший рядок повідомлення СУБД.

Результат: файл перевірок і розділ 9 звіту.

Крок 8. Перевірка тому

- Зупинити середовище командою `docker compose down` і запустити знову командою `docker compose up -d`.
- У pgAdmin оновити дерево об’єктів: таблиці схеми мають лишитися на місці.
- Команди і результат записати в протокол запуску.

Крок 9. Цикл up - down - up у порожній базі

- У pgAdmin створити порожню базу `dbsa_check` (Databases - Create - Database) і відкрити для неї Query Tool.
- Виконати всі файли `*.up.sql` з каталогу `migrations` за зростанням номерів.
- Виконати скрипт перевірки схеми (`lab-01/check-schema.sql`). Скрипт лише читає системний каталог і показує для кожної таблиці стовпці з типами та обмеження; рядок «УВАГА» означає таблицю без первинного ключа. Звірити результат з діаграмою реляційної схеми: ті самі таблиці, стовпці, типи, ключі й реакції на видалення. Розбіжності виправити новою міграцією або виправленням діаграми.
- Виконати всі файли `*.down.sql` за спаданням номерів і знову виконати скрипт перевірки схеми: результат має бути порожнім.
- Ще раз виконати всі файли `*.up.sql` і скрипт перевірки схеми: результат має збігатися з першим.

- Скопіювати в розділ 10 звіту результат першого виконання скрипта разом із заголовками стовпців і записати, чим завершилися два наступні. Після перевірки базу `dbsa_check` можна видалити.

Результат: розділ 10 звіту.

Структура каталогу і здача через Git

```
<репозиторій студента>/
├── README.md                              опис предметної області (лекція 1)
├── compose.yaml                           файл опису середовища, спільний для робіт
└── lab-01/
    ├── README.md                          звіт за шаблоном
    ├── constraint-checks.sql              перевірки обмежень
    └── migrations/
        ├── 001_create_schema.up.sql       створення схеми
        └── 001_create_schema.down.sql     скасування створення схеми
```

- Звіт - файл `lab-01/README.md`, створений зі шаблону звіту (`lab-01/report-template.md`). Діаграми - блоки `mermaid` у звіті; окремих файлів діаграм немає.
- Назви файлів міграції: тризначний номер, підкреслення, короткий опис англійською малими літерами і суфікс `.up.sql` або `.down.sql`. Кожна міграція - обов'язково пара файлів з однаковим номером і описом. Нумерація наскрізна для всіх робіт: наступна зміна схеми, навіть у роботі №2, отримає номер 002 і лежатиме в каталозі `migrations` тієї роботи.
- Файли міграції, що потрапили в коміт, не редагують; зміна схеми - нова пара файлів.
- Робота виконується кількома змістовними комітами, а не одним наприкінці: щонайменше окремі коміти для бізнес-правил, ER-моделі з ключами, нормалізації, реляційної схеми і міграції з перевірками. Повідомлення коміту описує зміну: «Додано ER-модель і таблицю ключів», а не «update».
- Історія комітів є частиною оцінювання: вона підтверджує самостійність роботи і дотримання правила про файли міграції.
- На перевірку подається стан репозиторію в основній гілці. Терміни здачі і спосіб повідомлення про готовність роботи оголошує викладач.

Зміст звіту

Звіт `lab-01/README.md` містить розділи в такому порядку (усі вони вже є в шаблоні звіту (`lab-01/report-template.md`)):

1. Предметна область - короткий опис: що це за бізнес, які події відбуваються в часі, які рішення в ньому ухвалюють.
2. Бізнес-правила - таблиця з кроку 1.

3. ER-модель - діаграма Mermaid, перелік сутностей, зв'язки реченнями в обидва боки, відмітка про перевірку за чотирма типовими помилками; за потреби - опис узагальнення.
4. Ключі та реакції на видалення - таблиці з кроку 3.
5. Нормалізація - функціональні залежності, «погана» таблиця, три аномалії, кроки 1NF, 2NF, 3NF із знятими проблемами, звірка з ER-моделлю, денормалізація або запис про її відсутність.
6. Реляційна схема - діаграма Mermaid з англійськими назвами, типами PostgreSQL і ключами.
7. Середовище - протокол запуску: операційна система, команди, вивід `docker compose ps`, параметри підключення pgAdmin (без пароля), результат перевірки тому.
8. Створення схеми - посилання на файли міграції і рішення, яких не видно з діаграми.
9. Перевірка обмежень - таблиця «правило - вид обмеження - повідомлення СУБД» і посилання на файл перевірок.
10. Цикл up - down - up - результат скрипта перевірки схеми, підтвердження порожнього результату після down і збігу після повторного up, висновок звірки з діаграмою розділу 6.

Критерії приймання

- У репозиторії є `compose.yaml` у корені, а в каталозі `lab-01` - звіт з усіма розділами, файли міграції `001_create_schema.up.sql` і `001_create_schema.down.sql` і файл перевірок обмежень.
- Бізнес-правил не менше восьми; для кожного вказано, де воно перевіряється; схемою перевіряється щонайменше п'ять правил.
- ER-модель має 6-10 сутностей; кожен зв'язок має кардинальність і обов'язковість участі; зв'язків «багато до багатьох» без асоціативної сутності немає; перевірку за чотирма типовими помилками відмічено.
- Для кожної сутності обґрунтовано первинний ключ; для кожного зв'язку «один до багатьох» обґрунтовано реакцію на видалення з погляду бізнесу.
- Функціональних залежностей не менше восьми; три аномалії показано конкретними діями; для кожного кроку нормалізації є отримані таблиці і знята проблема; підсумкові таблиці в 3NF або денормалізацію обґрунтовано письмово.
- Назви таблиць, стовпців і файлів міграції - англійською, малими літерами, через підкреслення, без лапок і транслітерації.
- Файл up виконується без помилок у порожній базі; кожна таблиця має первинний ключ; кожен зовнішній ключ має явний ON DELETE; обмеження відповідають таблиці бізнес-правил.
- Цикл up - down - up проходить без помилок: після down скрипт перевірки схеми повертає порожній результат, після повторного up результат збігається з першим.
- Результат скрипта перевірки схеми не містить рядків «УВАГА» і збігається з діаграмою реляційної схеми: ті самі таблиці, стовпці, типи і ключі.
- Перевірок обмежень не менше п'яти (NOT NULL, UNIQUE, CHECK, REFERENCES, первинний ключ); кожну команду СУБД відхиляє з очікуваним повідомленням.

- Протокол запуску і результат перевірки тому є у звіті.
- Діаграми Mermaid відображаються у вебінтерфейсі сервісу репозиторію без помилок.
- Історія Git складається з кількох змістовних комітів; файли міграції 001 після першого коміту не змінювалися.
- На захисті студент відповідає на контрольні питання на прикладі власної схеми.

Контрольні питання до захисту

1. Покажіть на своїй ER-діаграмі зв'язок «один до багатьох» і сформулюйте його реченням в обидва боки.
2. Чому для обраної викладачем сутності первинним ключем зроблено саме цей атрибут? Що станеться, якщо значення природного ключа зміниться?
3. Покажіть у своїй «поганій» таблиці часткову і транзитивну залежність. Яку нормальну форму порушує кожна з них?
4. Яку реакцію на видалення обрано для названого викладачем зв'язку і що станеться з пов'язаними рядками під час видалення?
5. Назвіть бізнес-правило своєї області, яке не перевіряє схема, і поясніть чому.
6. Чому таблиці у файлі up створено саме в такому порядку, а у файлі down видаляються у зворотному?
7. У закоміченому файлі `001_create_schema.up.sql` знайдено помилку. Як її виправити і чому не можна просто змінити цей файл?
8. Навіщо писати файл down, якщо схему можна просто видалити разом із базою?
9. Що станеться з вашою базою після `docker compose down`? А після `docker compose down -v`?
10. Знайдіть у результаті скрипта перевірки схеми обмеження, яке реалізує назване викладачем бізнес-правило.
11. Чим ER-модель розділу 3 відрізняється від реляційної схеми розділу 6 і від файлів міграції?

Типові помилки та як їх уникнути

- Windows: команди `docker` завершуються помилкою з текстом `open //./pipe/dockerDesktopLinuxEngine` або `docker version` не показує розділ Server. Протидія: запустити Docker Desktop і дочекатися стану «Engine running».
- Windows: Docker Desktop повідомляє, що віртуалізацію не виявлено або WSL не встановлено. Протидія: увімкнути віртуалізацію в BIOS/UEFI, виконати в PowerShell від імені адміністратора `wsl --install --no-distribution` або `wsl --update` і перезавантажити комп'ютер.
- Контейнер не запускається з повідомленням `port is already allocated` або `ports are not available` для порту 5432. Найчастіша причина у Windows - раніше встановлений PostgreSQL, що працює як служба. Протидія: перевірити, хто займає порт, командою `netstat -ano | findstr :5432` і зупинити службу PostgreSQL у «Службах» Windows або замінити ліве число в рядку порту файлу опису на 5433 і вказати цей порт у pgAdmin. Якщо

порт вільний, але помилка лишається, порт може належати до діапазону, зарезервованого Windows (перелік показує `netsh interface ipv4 show excludedportrange protocol=tcp`), - тоді теж обирають інший порт.

- Після перезавантаження комп'ютера pgAdmin не підключається до бази. Протидія: запустити Docker Desktop і виконати в корені репозиторію `docker compose up -d`.
- Українські значення в SQL-файлах чи звіті відображаються пошкодженими. Протидія: зберігати файли в кодуванні UTF-8.
- Таблиці створено в службовій базі `postgres`, а не в `dbsa`. Протидія: перед виконанням файлу міграції перевірити назву бази в заголовку Query Tool.
- Дані зникли після `docker compose down -v` або після видалення тому в Docker Desktop. Протидія: для зупинки середовища - лише `docker compose down`; схема відтворюється з файлів міграції, тому головна копія схеми - репозиторій.
- Таблицю створено через діалог pgAdmin із назвою з великої літери, і запит до неї завершується помилкою `relation ... does not exist`: pgAdmin записав назву в лапках. Протидія: таблиці створюються лише командами з файлу міграції, назви - малими латинськими літерами без лапок.
- Назви записано транслітерацією (`klient`, `data_vydachi`) або змішано мови. Протидія: для кожної сутності й атрибута підібрати англійське слово; вміст рядків при цьому лишається українським.
- Команда з REFERENCES завершується помилкою `relation ... does not exist`. Протидія: у файлі up спершу створювати таблиці, на які посилаються.
- Файл down завершується помилкою `cannot drop table ... because other objects depend on it`. Протидія: у файлі down видаляти таблиці у зворотному порядку створення.
- Файл down не скасовує все, що зробив up, і після циклу up - down - up схема відрізняється. Протидія: для кожної команди файлу up у файлі down має бути команда, що її скасовує; результат перевіряється скриптом перевірки схеми на кроці 9.
- Файл up завершився помилкою посередині, і повторний запуск падає на вже створених таблицях. Протидія: виправити файл і перевіряти його в новій порожній базі, як на кроці 9.
- Зв'язок «багато до багатьох» лишився нерозкладеним або атрибут зв'язку записано в одну із сутностей. Протидія: правило лекції 3 - якщо у зв'язку є власний атрибут, це асоціативна сутність.
- Первинним ключем обрано телефон, електронну пошту чи назву, яка може змінитися. Протидія: правило вибору сурогатного ключа лекції 3 і стовпець ідентичності лекції 5.
- 2NF перевіряють для таблиць з одноатрибутним ключем, або сурогатний ключ «приховує» залежність. Протидія: 2NF стосується лише складених ключів; залежності перевіряють за змістовними атрибутами, а не за службовим номером.
- Помилку виправляють у вже закоміченому файлі міграції. Протидія: нова пара файлів міграції з ALTER TABLE.
- Mermaid не малює діаграму: атрибут без типу, пробіл у назві, кома в типі. Протидія: тип - першим словом, слова назви через підкреслення, у діаграмі - `numeric` без параметрів; перед комітом переглянути звіт у вебінтерфейсі сервісу репозиторію.
- Перевірки обмежень виконано всім файлом, і видно лише першу помилку. Протидія: кожну перевірочну команду виконувати окремо.

Забезпечення роботи

- Файл опису середовища (`lab-01/compose.yaml`) - PostgreSQL 18.6 із томом і відображенням порту; копіюється в корінь репозиторію без змін, крім порту, якщо 5432 зайнятий.
- Шаблон звіту (`lab-01/report-template.md`) - копіюється як `lab-01/README.md`.
- Скрипт перевірки схеми (`lab-01/check-schema.sql`) - виконується в pgAdmin без змін.
- Програмне забезпечення: Docker Desktop (Windows, macOS) або Docker Engine із Docker Compose (Linux), у Windows - WSL 2; pgAdmin 4 версії 9.17; Git. Для виконання в аудиторії образ `postgres:18.6` варто завантажити заздалегідь командою `docker pull postgres:18.6`.
- Зразки діаграм і команд - колоди слайдів лекції 3, лекції 4 і лекції 5.
- Роздатковий матеріал лекцій: пам’ятка нотації Crow's foot і чек-лист чотирьох типових помилок (лекція 3), картка п’яти правил переходу (лекція 4), шпаргалка «правило - обмеження» (лекція 5).

Рекомендовані джерела

- Берко А. Ю. та ін. Кн. 1, розділи про моделювання даних і нормалізацію.
- Silberschatz A. та ін., розділи 6 (Database Design Using the E-R Model) і 7 (Relational Database Design); розділи 3-4 - команди створення таблиць і обмеження цілісності.
- Beaulieu A., розділи про створення бази даних та обмеження.
- Офіційна документація PostgreSQL, розділ Data Definition.
- Документація Docker, розділи Get started (зокрема встановлення Docker Desktop on Windows) і Docker Compose.